Decision-Native Architecture Canvas — Product & System Blueprint

Oct 17, 2026 · @Algo

1. How to use this document

Paste this document at the start of a new chat: it is the single source of truth for the project, so the new session builds on it instead of re-deciding it.

Every statement carries a tag so nothing is mistaken for fact:

- **Verified** means checked against a source on 2026-10-17, with the source linked. Re-check prices, licenses, and competitor features before relying on them.
- **Decision** means the founder has chosen it. Treat it as settled unless they reopen it.
- **Recommendation** means the author suggests it. It is not yet confirmed or tested.
- **Assumption** means unconfirmed. Ask or verify; never fill it in.

Rules for any assistant reading this:

- Do not invent prices, model names, rate limits, library versions, or competitor features. If a fact is not here and cannot be checked, say so.
- Founder priorities, in order: frontend and user experience, backend architecture, then simplicity over feature count.
- The founder dictated these ideas by voice, so a few words were transcribed unclearly. Where this document interprets one, it says so (see Open questions).

2. Product vision

The app is a paid web canvas where an engineer drafts or draws an AI-app architecture and gets context-aware guidance on every component choice: what to pick, why, what the alternatives are, and what would change the answer.

One-line pitch (working): every box on your architecture is a decision, and the app helps you make it well.

Decision. The frontend and user experience are the main strength of the product; backend architecture comes second.

Decision. The product is closed source and monetized. Open source was considered and rejected.

The problem it solves (founder's words, condensed): when people design a system they often do not know which option is best, what the alternatives are, or whether they have considered every factor, such as cost, latency, scalability, and metrics. A general AI chat tends to miss the range of possibilities and the pros and cons of each.

Two ideas at the core:

- **Guidance while drawing.** The user draws their own architecture. The app watches the drawing and the user's context, then suggests the best path, the alternatives, the pros and cons, and the reasons.
- **Debate for hard calls.** When a choice is unclear, several AI models argue the finalists and a judge returns one reasoned verdict, with the runner-up and what would change it.

What it is not: not just a diagram generator, and not a chat window. The canvas is the product; chat is a helper.

3. Competitive landscape (as of 2026-10-17)

Verified: products exist today for each ingredient of this idea, so the claim of having no competitor cannot be made. **Inference, not fact:** in this search no single product combined the user's context as a first-class input, per-component verdicts on a canvas, debate for close calls, and latency and cost numbers for AI apps. The search was limited, so treat this as a lead to re-check, not a finding.

Product	What it does, per its own pages	Overlap with this idea
Eraser	Generates diagrams from prompts, code, or images; can ask clarifying questions; connects to private Git repositories; supports reference libraries to ground output	Diagram generation and clarifying questions; no decision verdicts seen on the page

Product	What it does, per its own pages	Overlap with this idea
Jeda.ai	AI whiteboard with architecture diagram generation, a multi-model agent (the page lists 18 models), an AI Matrix for weighted trade-off grids and decision records, and anti-pattern analysis on a shared canvas	Closest overlap found: trade-off matrices and multi-model reasoning on a canvas
Architecto	Prompt-to-diagram for cloud architecture, with cost, threat, compliance, and documentation workflows (search snippet only)	Diagram plus cost estimate
IcePanel	C4-model collaborative modeling with as-is and to-be versions; AI features are opt-in and use OpenAI models (search snippets; see also the Thoughtworks Radar entry)	Versioned design states
verd	Command-line tool, usable through MCP and Slack, in which models from different families debate in roles across rounds before a stronger judge gives a verdict (search snippet)	The debate mechanism, without a canvas
ai-debate-tool	Command-line tool, Python API, MCP server, and VS Code extensions that debate Claude and Codex perspectives on code and architecture decisions, with a 0 to 100 consensus score; version 1.2.0 released 2026-01-03	The debate mechanism, without a canvas
MAD-Arch paper	Research proposal for multi-agent debate in architecture review, framed as decision support with a human in the loop and decision records (search snippet)	Academic support for the idea; not a product

Product	What it does, per its own pages	Overlap with this idea
Archgate	Turns architecture decision records into executable rules that run in CI and guide AI coding agents (search snippet)	Decisions as enforceable rules

Source quality: the Eraser, Jeda.ai, and ai-debate-tool pages were opened in full on 2026-10-17. The other rows rest on search snippets and must be re-checked.

What this means (Recommendation):

- Do not market the product as having no competitors. Position it as a decision-native canvas for AI-app architecture.
- Jeda.ai's page positions itself against AI that simply hands over one answer, and for AI that thinks with the user. Present verdicts as reasoned recommendations with flip conditions, not as a black-box answer.
- The defensible edge is execution: UX quality, an accurate maintained knowledge base, and depth in AI-app architecture.

Not verified: competitors' unreleased or paid features, their prices, whether Jeda.ai's matrices give context-specific verdicts, and anything released after this date.

4. Target user and launch wedge

Decision. Launch with AI-app architecture: agents, RAG, and LLM-powered apps. The founder is an AI engineer building such apps and wants to know where each component goes, which database or store to pick, where latency arises, and how scaling will play out later.

Recommendation (reasoning, not verified data): this niche is a good wedge because the tooling choices change quickly (models, prices, limits, vector stores, agent frameworks), builders in it make many first-time decisions, and generic chat advice goes stale. A narrow domain also keeps the knowledge base small enough to keep correct.

Assumption (unvalidated): AI engineers and small teams building AI products will pay for this. Test it with the Phase 0 blind test and a waitlist that shows a price.

Secondary users: freshers learning system design, and experienced engineers who want a second opinion. **Assumption:** freshers mostly use the free tier and act as the funnel; this is not validated.

Later domains (not now): general backend and infrastructure design, and front-end architecture.

5. Core experience

One engine serves two entry points, and both read the same context profile.

Mode A, design it for me. The app runs a short adaptive interview, then drafts an architecture on the canvas with a stated reason for every placement.

Mode B, I will draw it. The user starts on a blank canvas and draws. The app watches and guides. Users can start in either mode and switch; hand edits to a Mode A draft keep getting guidance.

Context profile (Decision): this is the first thing the app collects. It is shown as editable chips above the canvas, and every verdict updates when a chip changes.

Chip	Example values	Why it matters
Purpose	personal, startup, company	Sets how much rigor and cost is reasonable
Users now and in 12 months	about 500 now	Drives scale checks
Budget	free tiers only, small, flexible	Filters options by price
Deploy target	the hosting platform the user names	Limits options and shows free-tier fit
Database or stack already chosen	existing choices	Avoids recommending a rewrite
Team size and experience	solo, beginner	Favors simplicity
Must-haves	low latency, compliance	Hard constraints

The interview is 6 to 8 tappable cards. It is adaptive (an answer can trigger a follow-up, such as which free tiers the user already uses) and skippable.

Guidance while drawing (Recommendation):

- Quiet by default: a small badge on a node means a better fit exists for the user's context. Clicking it opens the drawer. No popups mid-stroke.
- Instant checks when a component is dropped or connected; deeper analysis when the user pauses, clicks a badge, or asks for a full review.
- The drawer shows the verdict, the alternatives, context-specific pros and cons, why each alternative lost, and what would flip the choice. A swap button replaces the node and rewires its connections.
- A what-next path suggests what is missing, for example no failure handling between an API and a database.
- Ghost suggestions: faded components on the canvas that one click makes real.
- A whole-design budget check for free-tier users: which component hits its limit first and what it costs after.

Typed palette (Recommendation). To judge a drawing, the engine must know what was drawn. The palette offers typed components (database, cache, queue, agent runtime, vector store, and so on) with real product options inside each type. Free-text custom boxes are allowed; the app tries to classify them and asks when unsure. The interface says what it could not interpret instead of guessing silently.

6. Feature scope

Scope is deliberately narrow. The founder asked not to complicate the product.

v1 at launch (Phases 1 and 2 of the roadmap):

- Context profile and adaptive interview
- Typed-component canvas with Mode A drafts and Mode B guidance badges
- Decision drawer: recommended option, alternatives, why-not, flip conditions
- Blind-spot view: what the design has not addressed yet
- Numbers strip for AI-app components: request latency breakdown and cost estimate
- Debate on demand for close calls
- Time slider: the architecture at today, 10x, and 100x load
- Export: a design document with decisions and reasons, plus decision records
- Accounts, saved projects with versions, usage metering

Later:

- Break-it mode: remove a component or multiply traffic and show what fails

- Team features: shared projects and review links
- Real-time multi-user editing
- More domains beyond AI apps

Out of scope for now:

- Scanning users' cloud accounts
- Terraform or other infrastructure-as-code generation
- A chat-first interface
- Covering every system-design factor; cover only what the knowledge base can keep correct

7. Decision engine

Each verdict comes from a staged pipeline in which deterministic steps do the elimination and the math, and language models only reason about what is left.

1. **Capture constraints** from the context profile: scale, budget, deploy target, existing stack, team, must-haves.
2. **Generate candidates** from the curated knowledge base, not from model memory, so options are not silently missed.
3. **Eliminate** candidates that break a hard constraint. Pure rules, no AI.
4. **Score** the survivors with transparent weights tied to the user's priorities.
5. **Debate the finalists** only when the top 2 or 3 are close or the user asks.
6. **Return the verdict:** the pick, why, the assumptions, what would flip it, the runner-up, and how costly the choice is to reverse.

Three cost layers, cheapest first (Recommendation):

Layer	What runs	When
1. Rules and calculators	Deterministic checks, latency math, cost math	Always, instantly
2. Small model	Explanations and alternatives for one component	On click
3. Debate with a strong judge	Several models argue the finalists	On request or for close calls

Starting rules for AI apps (to be refined): agent loop running inside a web request; no fallback model; unbounded context growth; single point of failure; read-heavy path with no cache.

Honest limits (Recommendation): a verdict is conditional on the user's constraints and never claims a universal best. For small projects the right answer is often that several options work and the user should pick the one they already know. Layer 1 is reproducible; model output in layers 2 and 3 is not, so each verdict stores its inputs, the knowledge-base version, and the model outputs that produced it.

Anti-hallucination rule (Recommendation, following the founder's no-hallucination requirement): prices, free-tier limits, rate limits, and product capabilities come only from knowledge-base records that carry a source and a verified date. A model may explain them but may not supply them. If a record is missing or stale, the app says so.

8. Multi-model debate and LLM providers

Decision. The debate uses models from more than one provider so one model's blind spots do not decide the verdict. The founder named ChatGPT (OpenAI) and, by voice, a second name transcribed as 'jev'. This document reads it as Gemini (Google). Claude (Anthropic) is a third candidate. **Open question:** confirm the intended providers.

Debate design (Recommendation):

- Only 2 or 3 finalists are debated, never the full option list.
- Each finalist gets an assigned advocate model. Assigned advocacy produces sharper arguments than asking models to agree politely.
- Supporting roles: a skeptic, a cost-and-operations voice, a security voice.
- A judge model, the strongest available at build time, must cite the user's context chips and the knowledge-base records in its verdict.
- Every turn returns structured output so the interface can render it. A fixed round cap keeps cost bounded. **Assumption:** 2 to 3 rounds, to be set by testing.
- The transcript, participants, and cost are saved with the verdict.

Not yet known: whether debate beats a single well-prompted model on this task. The Phase 0 blind test exists to answer that before the canvas is built. Related prior art exists (terminal debate tools and a research proposal, section 3), but none of it is evidence that debate improves accuracy here.

Provider access: gateway or direct (Open decision). Verified from OpenRouter's comparison page (vendor-authored, search snippet):

- OpenRouter is a hosted gateway: the app calls one API, and OpenRouter routes and fails over to upstream providers, with nothing for you to run.
- LiteLLM is an open-source proxy that you host yourself; its example setup uses Docker and PostgreSQL, and request data stays inside your network.
- LiteLLM can use OpenRouter as an upstream provider.

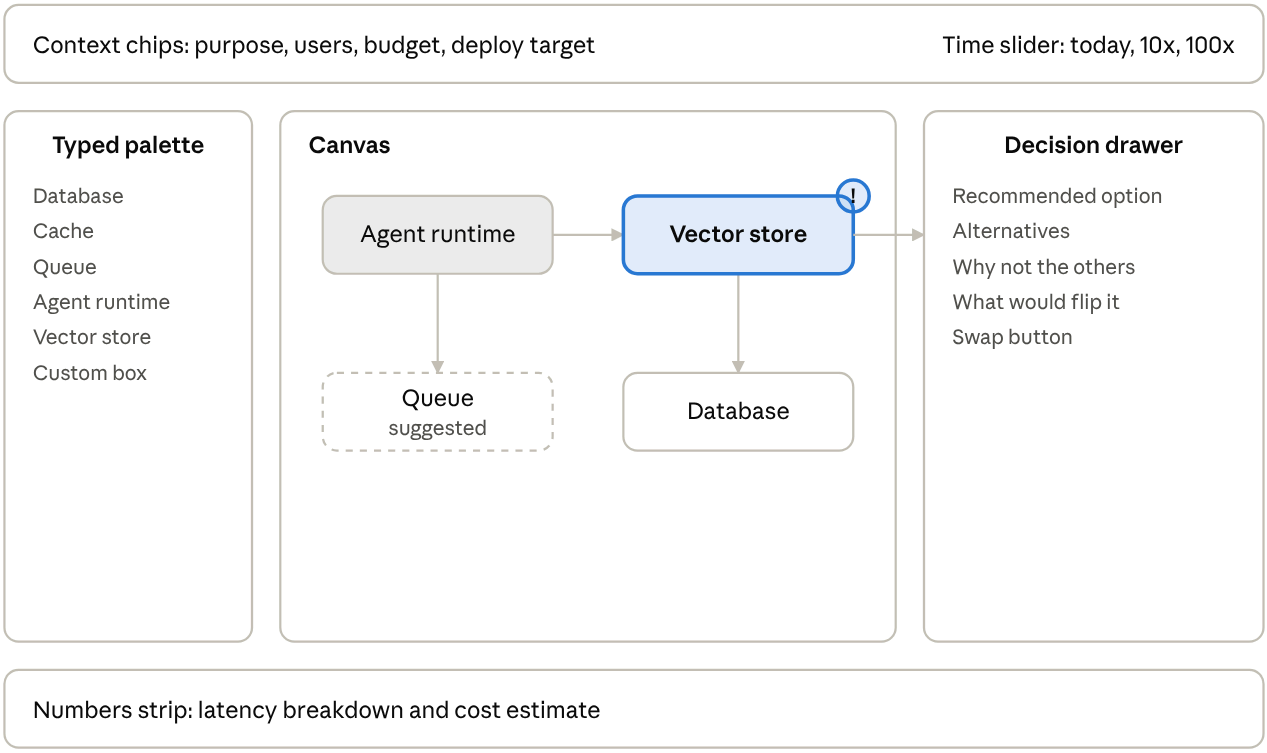
Recommendation: keep our own thin provider interface in the code either way, so the gateway choice stays swappable.

Model names and prices: not fixed here. Pin exact model IDs and read current prices from each provider's documentation at build time; the model landscape changes too fast to hard-code from this document.

9. Frontend architecture

The frontend is the main product surface, so it gets the first and largest share of design effort: a canvas with guidance beside it, never on top of it.

The canvas screen puts guidance beside the drawing, not on top of it



Solid: decided. Dashed: suggested. Hollow: undecided. Badge: at risk or a better fit exists.
The accent node is the one open in the drawer.

canvas screen layout · 5 regions

The canvas is the center of the screen; the drawer and the numbers strip react to whichever component is selected.

Principles (Decision): canvas first with chat as a helper; guidance is quiet and never interrupts; a premium, fast, keyboard-friendly feel; plain-language reasons everywhere.

Other screens (Recommendation):

- **Start:** one text box plus the context chips, or a skip to a blank canvas.
- **Compare view:** two or three options side by side across cost, latency, scale, and simplicity.
- **Debate theater:** finalists and roles argue in view, and the judge's verdict lands at the end.
- **Export:** the design document and decision records.

Canvas library (Recommendation, to be confirmed by a prototype). Three candidates and what was found:

Library	License facts	Fit for this app
React Flow	Verified: MIT licensed. The maintainers ask commercial users to support them through a paid Pro plan or sponsorship, which does not change the license	Its node-and-edge model matches typed components and our graph data model (author's judgment)
tldraw SDK	Verified from the docs: production use needs a license key. Commercial use needs a commercial license, which is annual with pricing set by sales. A free 100-day trial exists. Hobby licenses are non-commercial and keep a watermark	A freeform whiteboard, so a looser fit for typed nodes, plus a recurring license cost for a paid product
Excalidraw	Reported as MIT through its npm package (third-party page, not verified at the source)	Hand-drawn whiteboard feel; not evaluated further

Recommendation: React Flow, because the app's data is a typed graph rather than free drawing. Confirm with a one-week prototype that custom nodes, the drawer, and the badges feel good before committing.

Real-time transport: WebSocket or SSE (the founder's question). Findings from Stream's comparison and a production streaming guide (search snippets):

- Server-Sent Events (SSE) carry data one way, from server to browser, over plain HTTP. WebSockets are two-way.
- Major LLM providers' streaming APIs use SSE, and it is the common default for streaming model output. WebSockets are recommended when the client must also stream input mid-flight, such as voice or live collaborative editing.

This app's real-time needs are debate turns, verdict progress, and cost updates, which all flow from server to client. Drawing actions and button clicks are ordinary requests.

Recommendation: use SSE for v1 and revisit WebSockets only when real-time multi-user editing ships. A debate would start with a normal request that returns a job id, and the browser would then open a stream for that job.

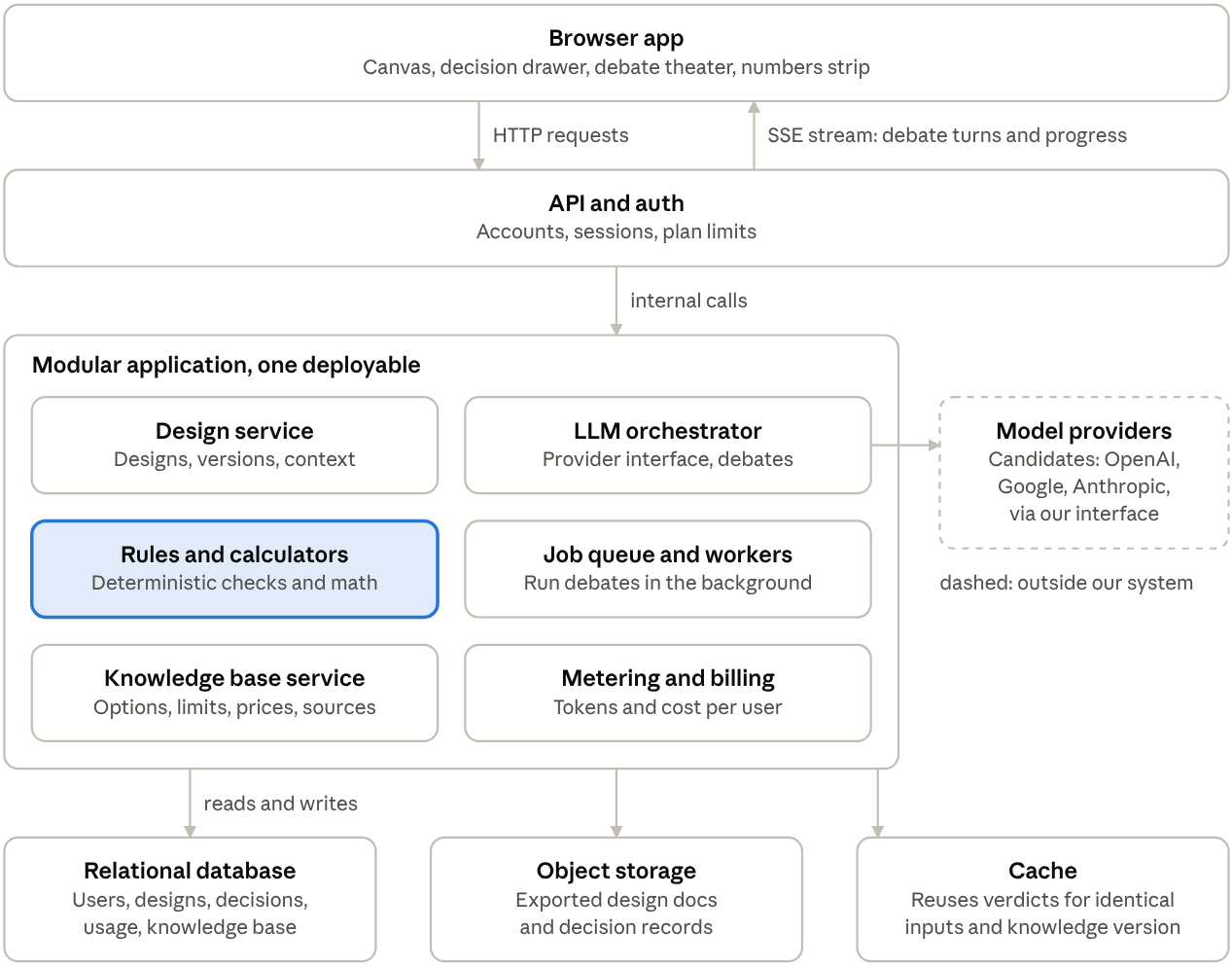
Pitfalls to design for, from the same guides: middleware that buffers a streaming response breaks streaming; when the browser disconnects, the server must cancel the upstream model call; send keepalives; and parse events as events rather than concatenating raw text.

Client state (Recommendation): canvas state lives in memory for speed and autosaves to the server as versioned snapshots. The screen is a view of the same structured design model the backend stores (section 11). Specific state-management and UI libraries are an **open decision**, to be made at build time.

10. Backend architecture and storage

Start as one modular application, not microservices, with deterministic code at its core and model providers as an external dependency.

A deterministic core with models at the edge



system architecture · 1 app, 3 stores, 1 external dependency

Rules and calculators are the core; model providers are reached only through our own interface, so a provider outage degrades debate and explanations but not the canvas.

Shape (Recommendation): the founder asked not to complicate the product, and a design tool should not over-engineer its own design. Split a module out only when load or team size demands it.

Modules:

- **API and auth:** accounts, sessions, plan limits.
- **Design service:** stores each design as a structured model with versions (section 11).
- **Rules engine and calculators:** deterministic checks, latency math, cost math.
- **Knowledge base service:** options, trade-offs, limits, prices, each with a source and verified date (section 12).

- **LLM orchestrator:** provider interface, role prompts, structured outputs, debate runner.
- **Job queue and workers:** run debates outside the web request, because they take time and cost money, and stream progress to the browser.
- **Metering and billing:** record every model call's tokens and cost per user from day one.

Storage (Recommendation, vendors not chosen):

- **One relational database** for users, designs, versions, decisions, debates, usage, and knowledge-base records. PostgreSQL is the default candidate.
- **Object storage** for exports.
- **A cache** keyed on a hash of the normalized constraints, the decision type, the options, and the knowledge-base version, so identical questions reuse earlier verdicts. A new knowledge-base version invalidates entries.

Hosting, queue technology, and billing provider: open decisions (section 16).

11. Data model

The canvas is a view of a structured design model. Analysis, debate, and exports all read the model, never the pixels. The field lists below are a starting draft (Recommendation), not a final schema.

Entity	Holds	Notes
Design	Owner, name, current version	One per project
DesignVersion	Graph, context profile snapshot, parent version, timestamp	Enables design A versus design B and the time slider
ContextProfile	Purpose, users now and in 12 months, budget tier, deploy target, existing stack, team, must-haves	Shown as chips
Component	Type, chosen product, parameters, state	Typed from the palette or classified from free text
Edge	From, to, sync or async, notes	Lets the rules see call chains

Entity	Holds	Notes
Decision	Component, options considered, verdict, rationale, assumptions, flip conditions, confidence, produced by (rules, small model, or debate), knowledge-base version	One per decided component
Debate	Decision, participants (provider and model), roles, rounds, transcript, verdict, cost	Linked from the decision
KnowledgeRecord	Type, option, trade-offs, limits, price, source URL, last verified, valid until, version	The only source of prices and limits
UsageEvent	User, kind, tokens, cost, timestamp	Feeds metering and plan limits

12. Knowledge base and freshness

The knowledge base is first-party and maintained by the founder's team, because the product is closed source and no community will keep it current.

Risk: stale data is the largest threat to trust. A wrong free-tier limit can produce a surprise bill for the user.

Rules (Recommendation):

- Every price, limit, and capability record carries a source URL, a last-verified date, and a valid-until date.
- The interface shows verified-on dates next to cost and limit figures and flags stale records.
- The knowledge base is versioned, and each verdict records the version it used.
- Re-verification is a scheduled process: automated checks can flag changes, and a human approves updates. The staleness threshold is an **open question**.
- Models never supply prices or limits (section 7).

Scope control (Proposal, not a decision): start with options for about 15 launch decision types in AI-app architecture, such as the model layer, agent runtime, vector store, memory and cache, queue, database, and observability.

13. Cost control and monetization

Decision. The product is closed source and paid. Price points are not set.

Cost driver: every debate is several model calls, so cost per user is real and must be metered from day one.

Levers (Recommendation):

- Use the three layers in section 7 so most guidance costs no model spend.
- Use cheaper models for explanations and the strongest model only for the judge.
- Cache verdicts for identical inputs.
- Cap debate rounds and finalists.
- Sell debates as metered credits.

Plan shape (Assumption, unvalidated): a free tier with one project and a few debates as the funnel, a paid individual tier, and a team tier with shared projects and review links. Free-tier limits and prices are undecided.

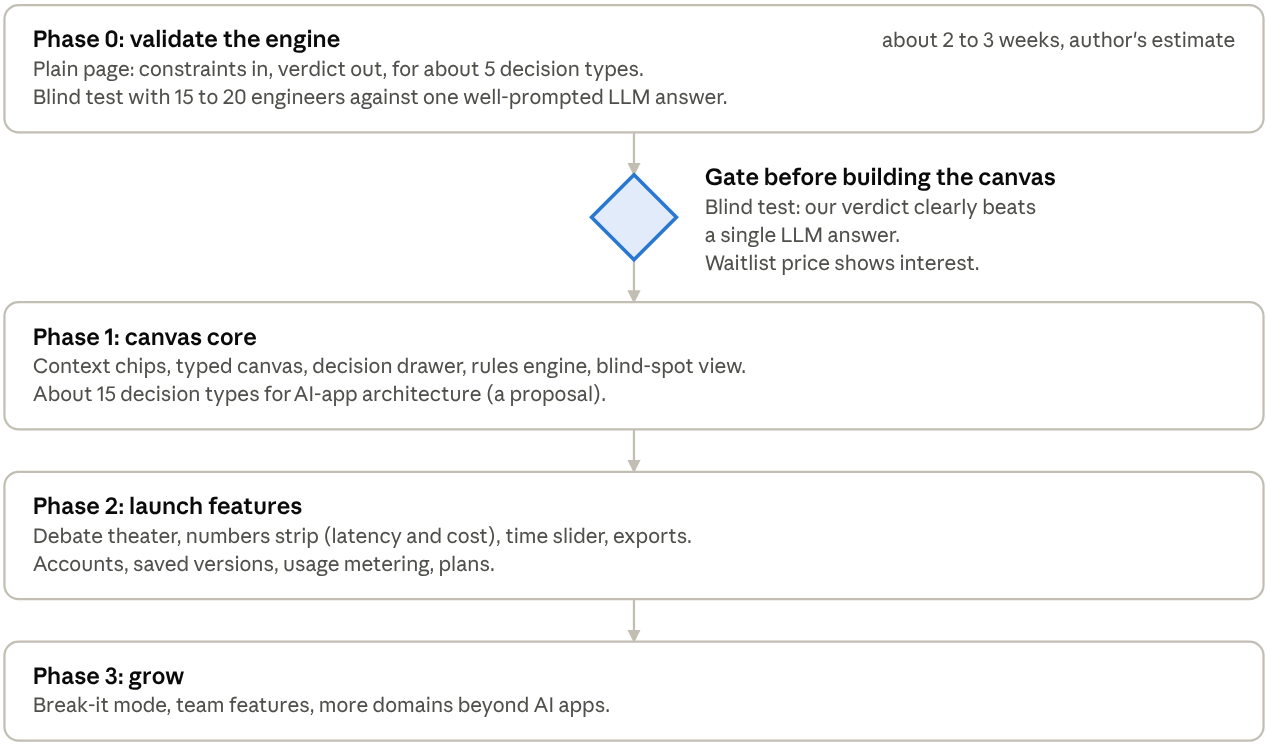
How to learn willingness to pay: a waitlist page that shows a price, run alongside the Phase 0 test.

Licensing cost to remember: if tldraw were chosen for the canvas, a commercial license is a recurring cost (section 9). React Flow is MIT licensed, and its maintainers ask commercial users for voluntary support.

14. Roadmap with validation gates

Build the decision engine on a plain page first, and only build the canvas if the blind test shows the verdicts are clearly better than a single model's answer.

One gate stands between validating the engine and building the canvas



roadmap · 4 phases, 1 gate

Durations are the author's rough estimates, not measurements. A polished v1 built solo and part-time was roughly guessed at three to four months, most of it spent on canvas UX. Phases 1 and 2 together make the v1 launch scope in section 6.

15. Risks and honest warnings

Risk	Why it matters	Mitigation
Confident wrong verdict	Users, especially beginners, cannot verify it	Conditional verdicts with assumptions and flip conditions; deterministic elimination; visible sources
Stale prices and limits	Can cause surprise bills and destroy trust	Section 12 process; verified-on dates; models never supply prices

Risk	Why it matters	Mitigation
Debate may not beat a single model	Unproven for this task	Phase 0 blind test before building the canvas
Models agreeing from shared bias	Agreement is not correctness	Assigned advocacy, different providers, grounding in the knowledge base
Debate cost	Several model calls per debate	Three layers, caching, round caps, credits
Easy to copy	A decisions panel and a debate prompt are cheap to replicate, and a multi-model AI whiteboard with trade-off matrices already exists	Compete on UX quality, knowledge-base accuracy, and AI-app depth; never claim to have no competitors
Positioning clash	A competitor's page positions itself against AI that hands over one answer	Present verdicts as reasoned recommendations with flip conditions
Scope creep	Covering every factor in all of system design never finishes	Narrow wedge (section 4) and the out-of-scope list (section 6)
Canvas UX is expensive	The polished canvas is the largest build cost	Use a canvas library; prototype early
Willingness to pay unknown	Free users may not convert	Waitlist with a price; metered credits
Output not reproducible	Model output varies between runs	Store inputs, versions, and outputs with every verdict

16. Open questions and pending decisions

- [ ] Confirm the second provider the founder dictated as 'jev' (read here as Gemini) and whether Claude is included.
- [ ] Gateway or direct provider SDKs for model access (section 8).
- [ ] Canvas library: confirm React Flow with a one-week prototype (section 9).
- [ ] Hosting platform, queue technology, database vendor, and billing provider.
- [ ] Product name.

- [ ] Free-tier limits, paid price points, and the price to show on the waitlist.
- [ ] Which decision types ship first (proposal in section 12) and which five go into the Phase 0 test.
- [ ] Staleness threshold for knowledge-base records, and who re-verifies them.
- [ ] Debate round cap and number of finalists (tune by testing).
- [ ] First users: AI engineers shipping products, or learners. Section 4 assumes AI engineers.
- [ ] Team size and timeline. A polished v1 built solo and part-time was roughly guessed at three to four months; that was the author's estimate and was not measured.
- [ ] Re-verify competitor features and prices before launch. Items resting on search snippets were not opened in full: Architecto, IcePanel, verd, the MAD-Arch paper, Archgate, OpenRouter's comparison, the two SSE guides, and Excalidraw's license.

17. Starter prompt for a new chat

Paste this document first, then this message:

```
You are helping me build the product described in the document above. Treat it as the source of truth. Do not re-decide anything tagged Decision. Ask me about anything tagged Assumption or listed under Open questions instead of guessing. Do not invent prices, model names, limits, library versions, or competitor features; if you cannot verify a fact, say so. Frontend and user experience come first, then backend architecture; keep scope small. Today I want to work on: [PASTE THE TASK, for example: design the Phase 0 decision pipeline, or lay out the canvas screens].
```